Go Development in the Age of AI

Frank Schröder

https://getstream.io

Who am I

- Frank Schröder (@magiconair)
- Staff Engineer at Stream (getstream.io)
- Writing Go apps since 2011

The Hard Truths

- The world as we know it is gone
- It is OK to mourn
- We are not doomed

The Past

- We turned vague ideas into code
- We thought through what we typed
- Writing the code and getting it to work was the interesting part

The Past

- Planning
- Typing, Reading, Understanding Code
- Debugging
- Risk assessment, Meetings

The Past

- For every tool that shipped, ten didn't
- Not bad ideas — too expensive to maintain
- The more senior you got, the clearer the cost

The Past

- January 2026: "Software Engineering is dead"
- One month of panic
- The signal took a while to find

The Present

- **Example 1: Bare metal CI cluster**

  15 nodes · different HW & locations ·

  puppet/Tailscale/k3s/iptables ·

  40 people use this every day (a lot)
- Built in days, not weeks
- Deploy, debug, fix, release, repeat

The Present

- **Example 2: Go deployment tool**

  ssh/EC2/SSM/ALB/puppet/github integration · ncurses+web UI · type down filtering · debug logging · 5 x faster deployments · 40 people use this every day
- 50 releases in 6 weeks (~2 per day)
- Debug, fix, release, repeat

The Present

- **Example 3: refactor repo with 1M LoC**

  break monolith into smaller parts
- Two failed attempts, should work with AI but didn't
- Task is too complex, needs steering, risk is too high

The Present

- **Example 4: move infra from cloud A to G**
  and from bare metal to k8s at the same time
- Servers, databases, load balancers, firewalls, migrations, alerts, metrics
- One person can build this, second person helps
- Took only some weeks from PoC to Ready to test

The Present

- Not 10x — 100x faster
- Days to minutes, weeks to hours
- It's transformational

The Present

- One-shot is a myth
- The right prompt → perfect app is luck, not process
- Can't be taught at scale

The Present - My Approach

- The Virtual Team Model
- Architect/PM · Dev team → You
- You stay in the driver's seat

The Present - My Approach

- The architect: probe the idea before writing a line
- The dev team: executes, never tires, never pushes back
- You: what to build, when to stop, when it's wrong

The Present - Workflows are lagging behind

- Most of the code is now AI-generated
- Too much AI code to review
- Tickets, PRs, and code reviews are for humans
- That mismatch is already here. We don't have a replacement yet

The Present - How to deal with this

- Build smaller chunks
- Verify behavior, not code
- Add more defensive code (cheap to add)

The Present

- Code hygiene still matters
- Stop and look at the structure
- Use your experience to redirect
- AI refactors in seconds

The (near) Future

- We turn vague ideas into features and systems
- First principles matter more, not less
- Scalability · latency · decomposition
- "How it works" is more important than the code
- You're now steering more complex systems

The (near) Future

Why Go still matters

- explicit over magic
- fast feedback loops
- right domain
- Cross-platform
- simple toolchain
- AI-generated Go reads like what it does

The (near) Future

What's now in reach

- Features for one customer
- Tools for three people
- Parallel rebuilds of entire systems
- Complex refactors
- Systems you couldn't justify before

The (near) Future

- We are early in the transition
- Old workflows don't fit · New ones aren't established yet
- The processes will catch up

The (near) Future

- Liberation, not death
- The craft changed
- The experience did not

Thank you

Built with AI and the virtual team model

https://getstream.io/

Questions?

Built with AI and the virtual team model

https://getstream.io/